PROGRAMACIÓN AVANZADA · SECCIÓN B · UNJBG · 2026-II

BUBBLESORT
y
MERGESORT

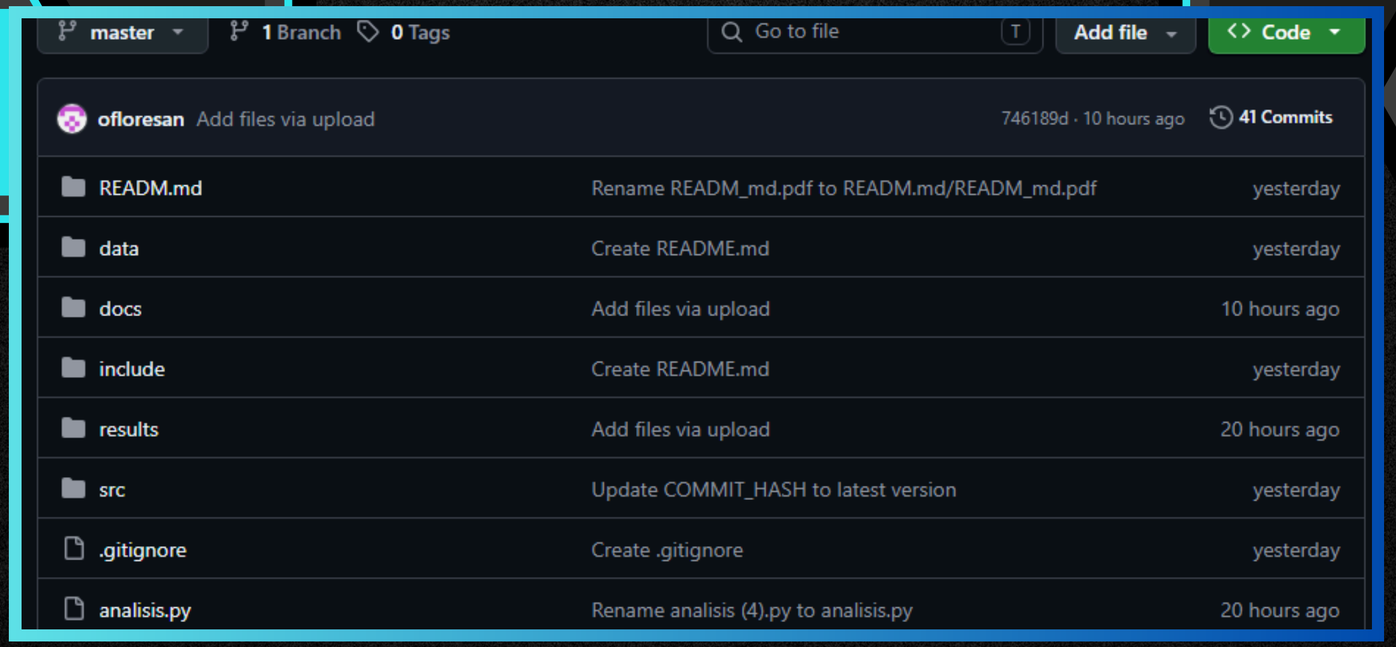
Comparación experimental del rendimiento bajo distintos escenarios de entrada

OBJETIVO

- Comparar el rendimiento de dos algoritmos de ordenación en C++
- Relacionar los resultados con su complejidad teórica

ALGORITMOS SELECCIONADOS

- Bubble Sort: directo, $O(n^2)$, memoria $O(1)$
- Merge Sort: logarítmico, $O(n \log n)$, memoria $O(n)$
- Ambos estables



ALGORITMOS

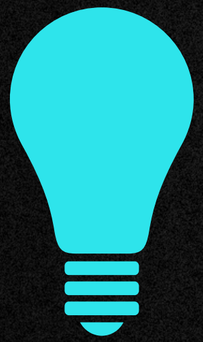
interDirectoDer (burbuja)
mezclar y dividir (Merge Sort)
Hechos a mano, sin funciones de la biblioteca

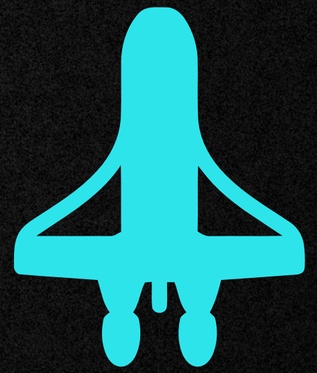
REGISTRO DE MÉTRICAS

- Comparaciones
- Intercambios o movimientos
- Profundidad de recursión
- Tiempo con steady_clock

BENCHMARK

- Generar(): 5 patrones con semilla
- Bucles: algoritmo, patrón, n y repetición
- Una fila de CSV por ejecución





```cpp
#include <fstream>
#include <chrono>
using namespace std;

// ================ CONFIGURACION ================
const int MAX_N = 100000;
const int REPS = 30;                    // repeticiones por condicion
const long long SEMILLA = 12345;        // semilla base
const char* COMMIT_HASH = "2c4e2dc";

// ================ VARIABLES GLOBALES ================
int A[MAX_N];                           // arreglo a ordenar
int aux[MAX_N];                         // arreglo auxiliar de Merge Sort
long long comparaciones, intercambios;  // contadores
int profundidad;                        // solo Merge Sort (en burbuja queda en 0)
long long semilla = SEMILLA;            // estado del generador aleatorio

// ================ BURBUJA (InterDirectoDer) ================
void interDirectoDer(int A[], int n) {
    comparaciones = 0; intercambios = 0; profundidad = 0;
    for (int i = 1; i <= n - 1; i++) {
        for (int j = 0; j < n - i; j++) {
            comparaciones++;
            if (A[j] > A[j + 1]) {
                int temp = A[j];
                A[j] = A[j + 1];
                A[j + 1] = temp;
                intercambios++;
            }
        }
    }
```

```
Bubble | aleatorio | n=1000 ...
Bubble | aleatorio | n=10000 ...
Bubble | ascendente | n=1000 ...
Bubble | ascendente | n=10000 ...
Bubble | descendente | n=1000 ...
Bubble | descendente | n=10000 ...
Bubble | casi_ordenado | n=1000 ...
Bubble | casi_ordenado | n=10000 ...
Bubble | duplicados | n=1000 ...
Bubble | duplicados | n=10000 ...
Merge | aleatorio | n=1000 ...
Merge | aleatorio | n=10000 ...
Merge | aleatorio | n=100000 ...
Merge | ascendente | n=1000 ...
Merge | ascendente | n=10000 ...
Merge | ascendente | n=100000 ...
Merge | descendente | n=1000 ...
Merge | descendente | n=10000 ...
Merge | descendente | n=100000 ...
Merge | casi_ordenado | n=1000 ...
Merge | casi_ordenado | n=10000 ...
Merge | casi_ordenado | n=100000 ...
Merge | duplicados | n=1000 ...
Merge | duplicados | n=10000 ...
Merge | duplicados | n=100000 ...

Listo. Errores de ordenacion: 0
Archivo generado: resultados.csv
```

01

- Reconstruir todo (Ctrl+F12)
- Ejecutar una prueba rápida (REPS = 2)
- Consola: avance por algoritmo, patrón y n

COMPILAR Y EJECUTAR

02

- 'Errores de ordenacion: 0'
- Abrir resultados.csv: tiempo, comparaciones e intercambios
- Mismos datos para ambos algoritmos

RESULTADOS DE ORDENACIÓN

03

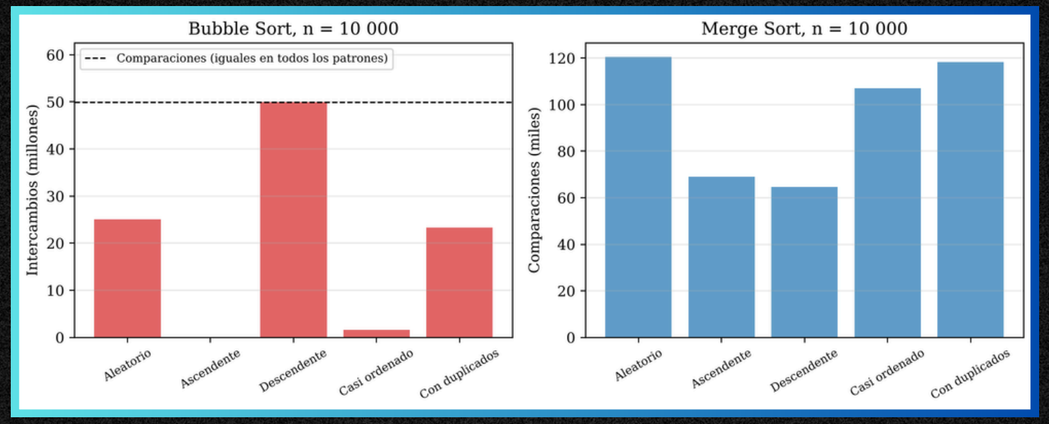
- 5 patrones de entrada
- n = 1 000 y 10 000 en la prueba rápida
- Corrida final: 30 repeticiones y 750 filas

DISTINTOS TAMAÑOS Y PATRONES



ANÁLISIS DE RESULTADOS

- Results/resultados.csv: datos crudos
- 4 tablas: tiempos y contadores de cada algoritmo
- 3 figuras: tiempo contra n, boxplot y contadores
- Analisis.py las genera



	patron	n	comparaciones	intercambios	profundidad
2	aleatorio	1000	499500	249025	0
3	aleatorio	10000	49995000	25015115	0
4	ascendente	1000	499500	0	0
5	ascendente	10000	49995000	0	0
6	descendente	1000	499500	499500	0
7	descendente	10000	49995000	49995000	0
8	casi_ordenado	1000	499500	16656	0
9	casi_ordenado	10000	49995000	1623316	0
10	duplicados	1000	499500	233856	0
11	duplicados	10000	49995000	23363148	0

	patron	n	comparaciones	intercambios	profundidad
2	aleatorio	1000	8712	9976	11
3	aleatorio	10000	120457	133616	15
4	aleatorio	100000	1536397	1668928	18
5	ascendente	1000	5044	9976	11
6	ascendente	10000	69008	133616	15
7	ascendente	100000	853904	1668928	18
8	descendente	1000	4932	9976	11
9	descendente	10000	64608	133616	15
10	descendente	100000	815024	1668928	18
11	casi_ordenado	1000	7163	9976	11
12	casi_ordenado	10000	107131	133616	15
13	casi_ordenado	100000	1403802	1668928	18
14	duplicados	1000	8588	9976	11
15	duplicados	10000	118268	133616	15

DIFERENCIAS ENTRE LOS ALGORITMOS

- Con n = 10 000, Bubble tardó 163 veces más (285,4 ms frente a 1,75 ms)
- Crecimiento compatible con $O(n^2)$ y $O(n \log n)$
- El patrón cambió los tiempos en un factor cercano a 2

CONCLUSIONES

- Merge Sort fue unas 163 veces más rápido con n = 10 000 y datos aleatorios.
- El crecimiento medido fue compatible con $O(n^2)$ y $O(n \log n)$.
- El patrón de entrada cambió los tiempos en un factor cercano a 2.
- La ventaja de Merge Sort cuesta memoria auxiliar de orden $O(n)$.